

Contents

PART 1: THE LANGUAGE

3	The first program
4	Values and variables
5	Deciding and repeating
6	Choosing: switch and enum
7	Functions and lists
8	Text and structs
PART 2: APPS	
10	The terminal
11	A game: snake
12	Digital rain
13	The internet
14	When it goes wrong
15	Reference
16	Programs to keep

Type along. Every example here runs as it is: save it with edit and run it with pico. The programs of part 2 are in ~/pico, to read and change. man pico is the short reference on the machine.

PART 1 OF 2 · THE LANGUAGE

pico.

the language of PocketType

```

1/4 17:29 73%
// The first program: run it with "pico hello.pico y
struct Point {
    int x;
    int y;
}

int dist2(Point p) {
    return p.x * p.x + p.y * p.y;
}

int main(str[] args) {
    str name = len(args) > 1 ? args[1] : "world";
    Point p = Point{x: 3, y: 4};
    for (int i = 0; i < 3; i++) {
        printf("hello %s %d, dist2 = %d\n", name, i
    )
    int[] squares = [];
    for (int i = 1; i <= 5; i++) {
        push(squares, i * i);
    }
    println("squares: ", squares);
    File f = open(getenv("HOME") + "/hello.log", "a"
28 lines L1/28 C1

```

C-like and typed, **safe**: a mistake stops the program at its line, never the machine.

Part 1 the language · **Part 2** apps: the terminal, games, the internet

1.0 beta 2

The first program

```

int main() {
    println("hello, pocket");
}

Write it with edit hello.pico, then Fn S to save and Fn Q to leave, and run it:
$ pico hello.pico
hello, pocket

```

pico compiles and runs in one go, `pico hello.pico` makes a program of it, `./hello`, that starts faster.

Every program has a `main`. What it returns is its exit status: 0 means all went well, `main` can have the words typed after the name:

```

int main(str[] args) {
    str who = len(args) > 1 ? args[1] : "you";
    println("hello, ", who);
    return 0;
}
$ pico hello.pico andre
hello, andre

```

Text and structs

```

str line = " the quick brown fox ";
str[] words = split(trim(line));
println(join(words, "-")); // the-quick-brown-fox
println(upper("hey"), find("hello", "ll"));

```

Also `substr`, `replace`, `contains`, `starts_with`, `repeat`, `chars`. `s[i]` is the byte at `i`, as a number.

STRUCTS

```

struct Player { str name; int x; int lives; }

Player p = Player{name: "ada", x: 1, lives: 3};
p.x += 2;
println(p); // Player{name: "ada", x: 3, lives: 3}

```

Structs, lists and text are passed by reference: a function that gets a `Player` changes that same `Player`.

FILES

```

File f = open("notes.txt", "w");
write(f, "a line\n");
close(f);
str all = read(open("notes.txt", "r"));

```

Values and variables

Every variable has a type, and says it:

```

int count = 3; // whole numbers
float price = 2.50; // with a point
bool done = false; // true or false
str name = "Ada"; // text
const int SIZE = 10; // never changes

```

TURNING ONE INTO ANOTHER

Nothing changes type by itself (except an `int` where a `float` is wanted):

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

`random(6)` is 0 to 5, `sqr`, `pow`, `sin`, `abs`, `min`, `max`.

```

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n // joins text, numbers too

```

SUMS

Also KEY_UP, KEY_DOWN, KEY_ESC; Enter comes as '\n'.

```
}
    print("\e[25H");
    raw_mode(false);
}
if (key == KEY_RIGHT) x++;
if (key == KEY_LEFT) x--;
if (key == 'q') break;
int key = readkey(100);
printf("\e[10;40H 0 ", x);
for(;;) {
    print("\e[2J\e[25T");
    raw_mode(true);
    int x = 10;
    while (iover) {
        int key = readkey(delay);
        dir = steer(key, dir);
        Point next = ahead(snake[0], dir);
        if (hits(next)) over = true;
        Insert(snake, 0, next);
        if (at(next, food)) grow();
        else erase(pop(snake));
    }
    Steering is a switch over the keys, and an enum names the way it goes:
    dir steer(int key, Dir now) {
        switch (key) {
            case KEY_UP: return UP;
            default: return now;
        }
    }
}
```

regardless.
gives KEY_NONE if nothing came, so the game moves on
raw_mode(true). readkey(100) waits at most 100 ms and
A game needs each key the moment it's pressed:

KEYS, AT ONCE

```
\e[31m \e[32m red, green text (30 to 37)
\e[1m \e[0m bold; back to normal
\e[2J clear the screen
\e[5;10H go to row 5, column 10
\e[25;10H hide, show the cursor
```

which are plain text:
term_rows(). Colours and the cursor are escape codes,
The screen is 53 columns by 23 rows (term_cols()),

The terminal

A game: snake



```
while (iover) {
    int key = readkey(delay);
    dir = steer(key, dir);
    Point next = ahead(snake[0], dir);
    if (hits(next)) over = true;
    Insert(snake, 0, next);
    if (at(next, food)) grow();
    else erase(pop(snake));
}
// the new head
// the tail
}
Steering is a switch over the keys, and an enum names the way it goes:
dir steer(int key, Dir now) {
    switch (key) {
        case KEY_UP: return UP;
        default: return now;
    }
}
```

pico ~/pico/snake.pico: the whole game is a loop that
reads a key, moves, and draws only what changed. In
outline:

The trick of any fast screen: build the frame as one text, write
it once, and write only the cells that change.
life.pico (Conway's life) and mandel.pico (a Mandelbrot
zoom) are next to it.

```
}
    print(frame);
    if (readkey(50) == 'q') break;
}
drops[x] = new_drop();
if (d.y - d.len >= rows)
    frame += at(d.y, x) + HEAD + glyph();
frame += at(d.y - 1, x) + GREEN + glyph();
d.y++;
Drop d = drops[x];
for (int x = 0; x < cols; x++) {
    str frame = "";
    struct Drop { int y; int len; int speed; }
}
```

pico ~/pico/matrix.pico: a drop for each column falls, a
bright head writing new glyphs, a trail fading behind it. In
outline:

Digital rain

ANY COMMAND
output("date") runs a command and gives back what it
printed; run("play", "x.mp3") runs one and lets it print.
str[] songs = split(output("ls", "/home/andre/music"))

```
$ pico ~/pico/weather.pico Roma
25.4 C, clear
humidity 50%, wind 11.9 km/h
```

A path is names and numbers joined by dots: for {"list":
{"name": "a"}, {"name": "b"}, {"name": "c"}}, "list" is a and "list.#"
counts the list.
println(json_get(j, "current.temperature_2m"));

```
str j = http_get("https://api.open-meteo.com/?
v1/forecast?latitude=41.9&longitude=12.5"
+ "&current.temperature_2m");
println(json_get(j, "current.temperature_2m"));
```

With Wi-Fi on, http_get(url) fetches what an address
answers, as text ("") if it fails). Web APIs answer in JSON, and
json_get(text, path) picks one value out of it.

The internet

When it goes wrong

The compiler says where, and often what to do:

```
hello.pico:4:13: error: expected int, got str;
parse it with to_int(s)
    int n = "12";
            ^
```

A mistake while it runs stops the program with the line, and
how it got there; nothing else is harmed:

```
list.pico:6: runtime error: index 9 out of
range (length 3)
    in main (list.pico:6)
```

FINDING MISTAKES

- println(x) show a value; lists and structs too
- assert(ok, "why") stop at once if something is wrong
- Fn C stop a program that runs away
- picoc -d x.pico the compiled instructions

TIPS

Start from a program in ~/pico and change it. Keep
functions short. Test on the PC too: make -C lang builds the
same pico there. docs/PICO-TUTORIAL.md and
docs/LANGUAGE.md have the rest.

Reference

```
int float bool str T[] File void
struct S { int a; } enum E { A, B }
const int N = 3; int[] a = [1, 2];
if else while for (;;) for (T x in a)
switch case default break continue return
?: && || ! + - * / % == != < > =
```

- print println printf format
- len of text, list
- push pop insert remove_at
- sort reverse slice find
- split join trim chars
- substr replace repeat
- upper lower contains
- starts_with ends_with
- to_int to_float str
- chr ord codes
- abs min max sqrt pow
- sin cos tan atan2 hypot
- floor ceil round exp ln
- random seed PI
- open close read write
- readline exists is_dir
- listdir mkdir remove
- rename stdin stdout
- run output getenv exit
- http_get json_get
- sleep_ms uptime_ms
- time date the clock
- raw_mode readkey
- term_cols term_rows
- assert checks
- KEY_UP ... KEY_ESC

pico f.pico runs, picoc f.pico compiles, man pico
reminds.

Programs to keep

In ~/pico on the machine, to run, read and take apart:

- matrix.pico digital rain, falling glyphs
- snake.pico the game: arrows or WASD
- weather.pico pico weather.pico Tokyo
- life.pico Conway's game of life
- mandel.pico a Mandelbrot zoom, in colour
- todo.pico a to-do list kept in a file
- guess.pico guess the number
- wc.pico counts lines, words, bytes
- hello.pico structs, arrays and a file
- benchmark.pico how fast the machine is

IDEAS TO TRY

A clock in big digits. A dice roller. Flash cards from a file in
~/notes. The day's news headlines from a JSON feed. A
pomodoro timer that runs beep with run(). A maze. A
calculator.

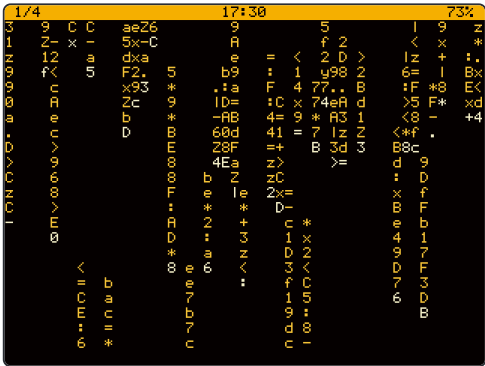
WHERE NEXT

man pico on the machine; on the PC, docs/PICO-
TUTORIAL.md to learn and docs/LANGUAGE.md for all of it:
every built-in, the limits, how the compiler works.

Run a program by its name. picoc snake.pico, put
snake in ~/bin, and snake is a command.

PART 2 OF 2 · APPS

pico.
the language of PocketType



Part 2 colours and keys, a game loop,
digital rain, the internet, commands,
mistakes, and a reference on the back.
All the programs are in ~/pico.

1.0 beta 2